

Symptom Description Routing for Public Health Information Retrieval Using Text Embeddings

May 7, 2026

1 Introduction

This project addresses the problem of routing short English symptom descriptions into broad public health information categories. The research question is: *Can text embeddings classify short symptom descriptions into broad health-information categories for routing users toward relevant public health resources?*

The system is designed for information retrieval and text classification education. It does not diagnose diseases, recommend treatment, provide medication advice, or perform emergency triage. The goal is to classify a short symptom description into one of six broad categories: respiratory, gastrointestinal, skin, neurological, musculoskeletal, and mental_health_sleep.

2 Dataset and Method

A custom dataset named `symptom_routing_expanded_dataset.csv` was created manually for this assignment. It contains 90 short English symptom descriptions with one label per example. The dataset was split into 67 training examples and 23 test examples using a 75/25 stratified split with random seed 712.

The project uses `nicher92/saga-embed_v1` as a frozen text embedding model. Each symptom description is converted into a dense embedding vector. Only the downstream classifiers were trained. This setup makes it possible to compare several lightweight classifiers on the same semantic representation.

Four classifiers were trained and evaluated:

- Logistic Regression
- Linear SVM
- KNN with cosine distance
- Random Forest

The pipeline is:

```
symptom text -> embedding model -> vector -> classifier -> category
```

3 Evaluation

The classifiers were evaluated on the held-out test set using accuracy, macro F1, and weighted F1. Macro F1 was included because it measures performance across all classes and is useful when evaluating whether the model performs consistently across categories.

Logistic Regression and Linear SVM achieved the same highest score. Logistic Regression was selected for the Hugging Face demo because it supports probability scores through `predict_proba`, making the demo output more informative.

Classifier	Accuracy	Macro F1	Weighted F1
Logistic Regression	0.957	0.952	0.957
Linear SVM	0.957	0.952	0.957
KNN cosine	0.783	0.763	0.764
Random Forest	0.913	0.903	0.909

Table 1: Classifier performance on the held-out test set.

4 Demo and Links

The final solution includes a custom dataset, trained model files, evaluation outputs, and a working Gradio demo hosted on Hugging Face.

- GitHub repository: <https://github.com/aa9911220/english-symptom-routing-embeddings>
- Hugging Face dataset: <https://huggingface.co/datasets/charlie0831/english-symptom-routing>
- Hugging Face model: <https://huggingface.co/charlie0831/symptom-routing-embedding-classifiers>
- Hugging Face demo Space: <https://huggingface.co/spaces/charlie0831/symptom-routing-embedding-demo>

The demo allows a user to enter a short symptom description and returns a predicted public health information category, a suggested retrieval focus, and category probability scores.

5 Limitations

This project is based on a small educational dataset with 90 manually written examples. It may not generalize well to real-world symptom descriptions, spelling mistakes, slang, or complex cases involving symptoms from multiple categories. The predicted labels are broad information-routing categories, not diagnoses or clinical conditions.

This model is for teaching information retrieval and text classification. It does not provide medical diagnosis, treatment advice, or emergency guidance.

6 Reflection on AI Tool Use

Using Codex and similar AI coding tools felt very powerful. At the same time, it did not feel like traditional programming, because many parts of the process could be completed by the AI after I described what I wanted. Instead of writing every line of code manually, I mainly gave instructions, checked the output, and decided what needed to be changed. This was surprising, because it showed how much of the programming workflow can now be assisted or automated by AI.

However, this also made it clear that understanding the project is still important. Even if the AI can generate code, I still needed to understand the dataset, the embedding model, the classifier pipeline, the evaluation metrics, and the safety limitations. I also needed to check that the project was framed as information routing rather than medical diagnosis. My impression is that this way of working may become an important trend in the future technology industry, where developers use AI tools to speed up implementation while still being responsible for verifying and understanding the final system.

References

- Hugging Face. <https://huggingface.co/>
- Gradio. <https://www.gradio.app/>
- Sentence Transformers. <https://www.sbert.net/>